

Architecture Optimization Series

2026 Spring

DeepSeek v4 混合注意力机制

CSA、HCA、MQA、分组投影与位置编码细节

Author: 喂喂薇

本讲摘要

本篇单独整理 DeepSeek v4 中混合注意力相关的笔记内容，重点包括 **CSA 压缩稀疏注意力**、**Lightning Indexer top-k 选择**、**HCA 与 Shared Key-Value MQA**、**Grouped Output Projection**，以及 Query/Key-Value 归一化、Partial RoPE 和 Attention Sink 等细节优化。

1 混合注意力: CSA 与 HCA

1.1 压缩稀疏注意力机制

Compressed Sparse Attention, CSA 的核心架构如图所示：它首先将**每 m 个 token 的 KV 缓存 (KV cache)** 压缩为一个条目 (entry)，再通过 **DeepSeek 稀疏注意力机制** 进一步加速计算。

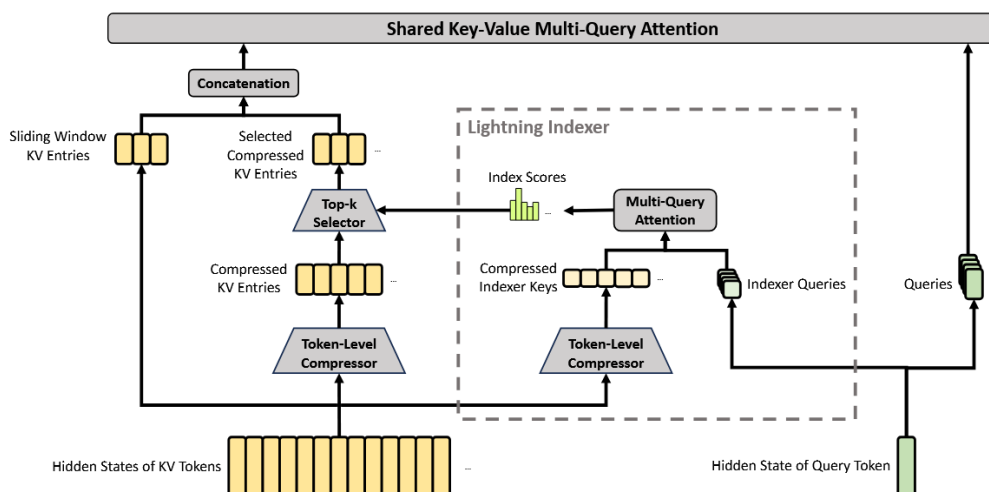


图 1: CSA 架构

CSA 中的 Token-Level Compressor

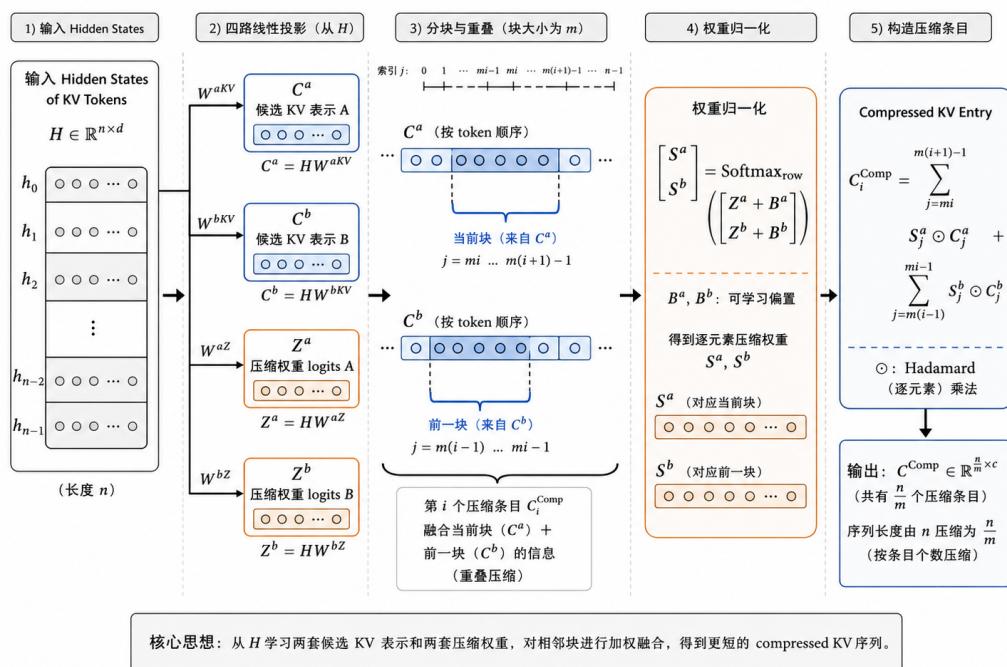


图 2: CSA 压缩原理

通俗理解版

CSA 的输入是 Attention 机制前的隐向量 H , 也就是 Transformer 架构把 token 转成的稠密向量。在标准的 Attention 机制下, H 会经过线性投影变成 QKV, 再完成后续操作。而在 CSA 里, H 会变成 KV 的混合表示, Q 正常投影, 最后进入下一个环节。

1.2 CSA 详细推导

设 $H \in \mathbb{R}^{n \times d}$ 为**输入隐藏状态序列**, 其中 n 为序列长度, d 为隐藏层维度。它就表示当前 Transformer layer 进入 attention 模块之前的 token 表示。

CSA 首先计算两组 KV 条目 $C^a, C^b \in \mathbb{R}^{n \times c}$ (**所谓 KV 条目也就是一对 KV 向量, CSA 里直接把 KV 融在一起了**) 及其对应的压缩权重 $Z^a, Z^b \in \mathbb{R}^{n \times c}$, 其中 c 为注意力头维度:

$$\begin{aligned} C^a &= H \cdot W^{aKV}, & C^b &= H \cdot W^{bKV} \\ Z^a &= H \cdot W^{aZ}, & Z^b &= H \cdot W^{bZ} \end{aligned}$$

为什么权重还要搞成学习的？

动态的，这样模型可以自己知道哪些 token 更重要，哪些 token 可以少占一点权重。

接下来， C^a 和 C^b 中每 m 个 KV 条目将依据其压缩权重和可学习位置偏置 $B^a, B^b \in \mathbb{R}^{m \times c}$ 被压缩为一个条目，最终产生压缩后的 $C^{Comp} \in \mathbb{R}^{\frac{n}{m} \times c}$ 。

每个压缩条目 $C_i^{Comp} \in \mathbb{R}^c$ 的计算方式如下：

$$\begin{aligned} [S^a; S^b] &= \text{Softmax}_{\text{row}}([Z_{mi:m(i+1)-1}^a + B^a; Z_{m(i-1):mi-1}^b + B^b]) \\ C_i^{Comp} &= \sum_{j=mi}^{m(i+1)-1} S_j^a \odot C_j^a + \sum_{j=m(i-1)}^{mi-1} S_j^b \odot C_j^b \end{aligned}$$

这个公式是说对参与压缩的 $2m$ 个候选 KV，计算一个归一化权重，然后用这些权重加权求和。即第 i 个压缩 KV，不只来自当前这一组的 m 个 token，还会额外融合前一组 m 个 token。 $\odot$ 即逐元素相乘，也就是每个元素对应的压缩权重相乘。

前面提到，原始序列长度一共是 n 个，对应 $C_0, C_1, C_2, \dots, C_{n-1}$ ；而压缩的过程中，会每 m 个 C_i 里压缩成 1 个，但整个过程要参考的量是 $2m$ ，存在重叠（也就是 1,2; 2,3; 3,4 这种，而不是 1,2; 3,4; 5,6 这种），所以最终实现的压缩效果是个数压缩了 m 倍。

注意是个数压缩了 m 倍，也就是说原来有 n 个 KV 候选条目 C ，现在变成 n/m 个了，而不是说每个 C_i^{Comp} 所蕴含的信息被压缩了。

举个例子

假设 $n = 12, m = 4$ ，则原始 KV 有 12 个：

$$C_0 \ C_1 \ C_2 \ C_3 \mid C_4 \ C_5 \ C_6 \ C_7 \mid C_8 \ C_9 \ C_{10} \ C_{11}$$

压缩后只有： $C_0^{Comp}, C_1^{Comp}, C_2^{Comp}$ 。

$$\begin{aligned}
C_0^{Comp} &\leftarrow \text{当前块 } C_0 C_1 C_2 C_3 + \text{前一块 padding} \\
C_1^{Comp} &\leftarrow \text{当前块 } C_4 C_5 C_6 C_7 + \text{前一块 } C_0 C_1 C_2 C_3 \\
C_2^{Comp} &\leftarrow \text{当前块 } C_8 C_9 C_{10} C_{11} + \text{前一块 } C_4 C_5 C_6 C_7
\end{aligned}$$

从 12 个变为 3 个，缩小了 m 倍。

1.3 top-k Query

接下来是 CSA 的第二步：**压缩完 KV 之后，怎么从这些 compressed KV entries 里面挑出最值得参与 attention 的 top-k 块。**

前一部分解决的是 $n \rightarrow n/m$ ，也就是**先把 KV 数量压短**。这一部分解决的是 $n/m \rightarrow k$ ，也就是在压缩后的 KV 里面，再挑出 top-k 个最相关的 compressed KV entries 参与真正的 core attention。

在 CSA 里，KV 已经不走标准的 Transformer 架构的线性投影了，但是 Q 还是走的（即 $Q = HW^Q$ ）。对于当前 query token，它不想让这个 token 对所有 compressed KV 都做 attention，而是希望在所有历史 compressed KV block 里，**挑选出最可能对当前 token 有用的 blocks**。

于是，设计出了一个 **Lightning Indexer**，给每个 compressed block s 算一个分数 $I_{t,s}$ ，然后选出分数最高的 top-k 个 compressed KV entries。最后得到 $C_t^{SprsComp}$ ，也就是当前 query token 真正要看的稀疏压缩 KV 集合。

具体的 Lightning Indexer 设计过程里，对于 query token t ，会先以**低秩 (low-rank)** 方式生成**索引器查询向量** $\{q_{t,1}^I; q_{t,2}^I; \dots; q_{t,n_h}^I\}$ ，其中 n_h 是 indexer query head 的数量。

什么是 low-rank 方式？

索引器查询向量不是直接从 h_t 一步投影出来的，而是分两步。

$$c_t^Q = h_t \cdot W^{DQ}$$

把 hidden state 从 d 维降到 d_c 维。

$$[q_{t,1}^I; q_{t,2}^I; \dots; q_{t,n_h}^I] = q_t^I = c_t^Q \cdot W^{IUQ}$$

再从 d_c 维升到所有 indexer query heads 拼起来的维度。

即整个过程是 $h_t \in \mathbb{R}^d \rightarrow c_t^Q \in \mathbb{R}^{d_c} \rightarrow q_t^I \in \mathbb{R}^{cn_h^I}$ ，**先降维再升维**。这种不用一个巨大的矩阵直接投影，而是用两个较小的矩阵分解它，减少参数数量和计算量的方式就是所谓的 low-rank 方式。

CSA 使用和生成 C^{Comp} 相同的压缩操作，得到压缩后的 indexer keys K^{IComp} 。之后就可以给 K^{IComp} 和索引器查询向量融合在一起了。

$$I_{t,s} = \sum_{h=1}^{n_h^I} w_{t,h}^I \cdot \text{ReLU}(q_{t,h}^I \cdot K_s^{IComp})$$

$$\text{ReLU}(x) = \max(0, x)$$

$$[w_{t,1}^I; w_{t,2}^I; \dots; w_{t,n_h^I}^I] = w_t^I = h_t \cdot W^w$$

$$C_t^{SprsComp} = \{C_s^{Comp} \mid s \in \text{Top-}k(I_{t,:})\}$$

其中 $q_{t,h}^I \cdot K_s^{IComp}$ 是一个点积，表示当前 token 的第 h 个 indexer query head 和第 s 个 compressed indexer key 的匹配程度。 K_s^{IComp} 和 C_s^{Comp} 是一一对应的， q 和 K 的匹配程度就代表 q 和 C 的匹配程度。ReLU 会把负数截断成 0，所以它只保留正相关的匹配分数，负相关不贡献。

$w_{t,h}^I$ 是给不同 indexer head 的一个融合权重，因为之前的多个头（对应多个检索视角）并非同样重要，所以搞 w 给不同头不同权重以示区分。

这样就计算好了最终 index score: $I_{t,s}$ 。最后，就可以按照 Top- k 来选分数最高的 k 个压缩后的 $C_t^{SprsComp}$ 了。

举个例子

假设 $n = 16, m = 4$ 。原始 token 分成 4 个 compressed block:

block 0: token 0 1 2 3 $\rightarrow C_0^{Comp}$

block 1: token 4 5 6 7 $\rightarrow C_1^{Comp}$

block 2: token 8 9 10 11 $\rightarrow C_2^{Comp}$

block 3: token 12 13 14 15 $\rightarrow C_3^{Comp}$

现在当前 query token 是 $t = 14$ ，它属于 $\lfloor 14/4 \rfloor = 3$ 。

对于当前 query token t ，它要给前面的 compressed block s 打分，即 $s < \lfloor t/m \rfloor$ 。这个约束是因果性的，也就是只看当前 token 之前的 compressed block，不看未来 block。

所以它只能看前面的 compressed blocks: $C_0^{Comp}, C_1^{Comp}, C_2^{Comp}$ 。Lightning Indexer 会算: $I_{14,0}, I_{14,1}, I_{14,2}$ 。假设算出来: $I_{14,0} = 0.2, I_{14,1} = 1.7, I_{14,2} = 0.9$ 。如果 $k = 2$ ，那 top-2 是 block 1, block 2，于是：

$$C_{14}^{SprsComp} = \{C_1^{Comp}, C_2^{Comp}\}$$

后续真正 attention 时, token 14 就只对这两个 compressed KV entries 做核心 attention, 而不是对所有历史 compressed entries 都做。

怎么理解 K^{IComp} 和 C^{Comp} , 它们是怎么对齐的?

C^{Comp} 可以理解为真正的内容, 用于真正 attention 读取。 K^{IComp} 是索引表示, 用于快速判断要不要看这个 block。

它们两者是同一个 Hidden 向量得到的。生成 C^{Comp} 时, 大概是 $H \rightarrow C^a, C^b$, $H \rightarrow Z^a, Z^b$, 然后每 m 个位置压成一个 C_s^{Comp} 。生成 K^{IComp} 时, 可以理解为类似 $H \rightarrow K^{Ia}, K^{Ib}$, $H \rightarrow Z^{Ia}, Z^{Ib}$, 然后也按同样的 token-level compression 规则, 每 m 个位置压成一个 K_s^{IComp} 。这样它们两个实际就有了对齐关系。

再举个例子: 假设 $m = 4$, 原始 token 是:

0 1 2 3 | 4 5 6 7 | 8 9 10 11 | 12 13 14 15

那么压缩后:

$$\begin{aligned} C_0^{Comp} &\leftrightarrow \text{block 0}, & C_1^{Comp} &\leftrightarrow \text{block 1}, \\ C_2^{Comp} &\leftrightarrow \text{block 2}, & C_3^{Comp} &\leftrightarrow \text{block 3}. \end{aligned}$$

同时 indexer keys 也是:

$$K_0^{IComp}, K_1^{IComp}, K_2^{IComp}, K_3^{IComp}$$

分别对应 block 0 到 block 3。所以有一一对应关系:

$$K_0^{IComp} \leftrightarrow C_0^{Comp}, \quad K_1^{IComp} \leftrightarrow C_1^{Comp}, \quad K_2^{IComp} \leftrightarrow C_2^{Comp}, \quad K_3^{IComp} \leftrightarrow C_3^{Comp}.$$

原始标准的 Transformer 结构里的 QKV 去哪了, 它和 C_s^{Comp} 是什么关系?

Q 还是正常的套路, 在图右侧也能看到。对于 KV 而言, 它们已经大变, 分成两类来源:

- 一类就是 C_s^{Comp} ;
- 另一类技术报告中没有展开讲, 是图最左侧的 **Sliding Window KV Entries**。

这两部分拼接到一起, 送到最终的 Attention 机制里。

图最左侧的 Sliding Window KV Entries 是什么？

这部分的作用是保留局部窗口内的原始细粒度 KV。因为压缩 KV 有一个问题：它把多个 token 合成一个 entry，长程信息可以压缩保存，但局部细节可能会丢失。

这部分就相当于之前的标准模式下 Transformer 对于 K、V 的线性投影操作，只是限制的 token 数量会比较少。

为什么既要压缩所有 token，又要单独保留局部的 uncompressed tokens？这不是又压缩又不压缩吗？

CSA 的压缩是针对整个历史序列（远程历史）做的，不区分局部或全局，所有历史 token 都被压缩到了压缩池 C^{Comp} 里。

Sliding Window KV Entries 并不是压缩这一部分的历史 KV，而是专门保留最近的 uncompressed token KV，用来保留局部精细依赖。具体实现上，就是划分一个窗口，例如 Sliding Window 长度 = 128 tokens。所有剩余历史 token 都归为“长程区域”。

1.4 HCA & Shared Key-Value Multi-Query Attention

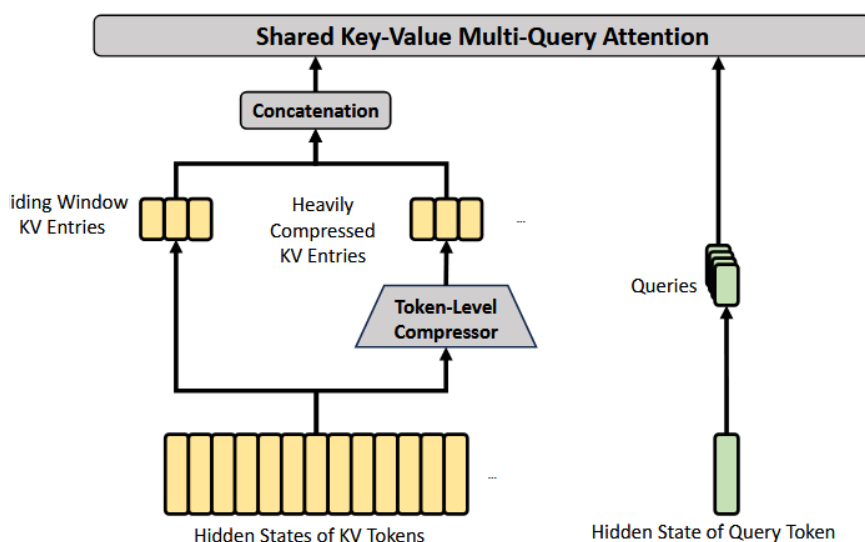


图 3: HCA 架构

HCA 和 CSA 流程基本上一模一样，区别在 HCA 压缩得更狠，也即 HCA 的压缩窗口 m' 比 CSA 的压缩窗口 m 大很多；而且 HCA 不做 overlap，也不使用 sparse

attention。

MQA 这部分对应的是 CSA 的核心 attention，也就是 **Lightning Indexer** 和 **token-level compression** 之后，真正参与预测的注意力。

1.4.1 MQA 过程

- **输入**：Query token h_t 的压缩表示 c_t^Q 生成多头 query：

$$[q_{t,1}, q_{t,2}, \dots, q_{t,n_h}] = q_t = c_t^Q W^{UQ}$$

这里 n_h 是 query head 数量。

- **Key/Value**：top- k 压缩 KV entries $C_t^{SprsComp}$ 。这部分已经由 Lightning Indexer 选出来，数量远小于原始序列长度。
- **Shared Key-Value**：每个 compressed KV entry C_s^{Comp} 同时作为 **Key** 和 **Value**。这与标准 Transformer 不同：标准里 $K \neq V$ ，而 CSA 这里共享 Key/Value 用于节省参数和计算量。
- **Multi-Query**：每个 query token 会生成 n_h 个 query head，分别与共享 KV 交互。

$$o_{t,i} = \text{CoreAttn}(q_{t,i}, \text{key} = C_t^{SprsComp}, \text{value} = C_t^{SprsComp})$$

输出：每个 head 得到 $o_{t,i}$ 的核心 attention 输出。

1.5 Grouped Output Projection: 分组投影

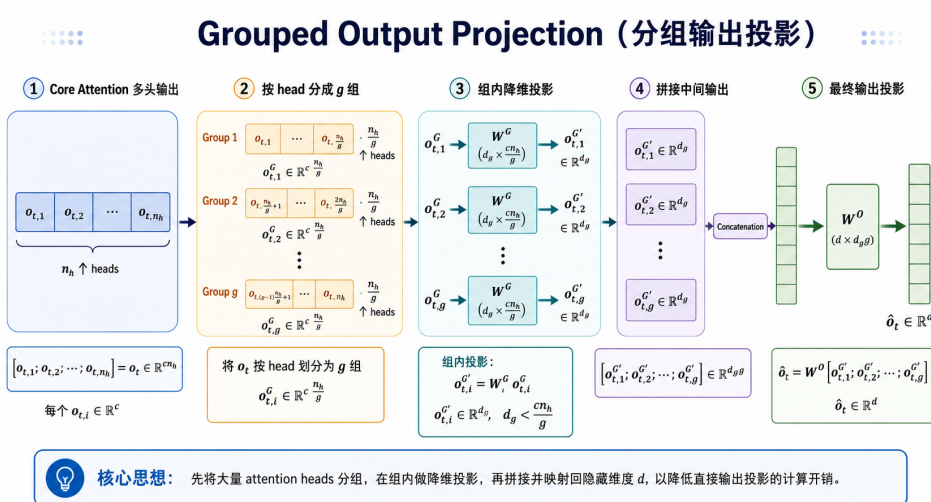


图 4: Grouped Output Projection 分组投影

在 DeepSeek-V4 中, query head 数量 n_h 很大, 如果直接把每个 head 的输出 $o_{t,i} \in \mathbb{R}^c$ 投影回 hidden state 的维度 d , 计算开销太大。因此引入分组投影策略。

- 把 n_h 个输出分成 g 组, 每组包含 n_h/g 个 head。
- 组内投影: 对每组的输出 $[o_{t,1}^G, o_{t,2}^G, \dots, o_{t, n_h/g}^G] \in \mathbb{R}^{c \cdot n_h/g}$ 进行投影到中间维度 $d_g < c \cdot n_h/g$ 。
- 拼接投影: 将各组的中间输出 $o_{t,1}^{G'}, \dots, o_{t,g}^{G'}$ 再拼接投影回原始 hidden state 维度, 得到 $\hat{o}_t \in \mathbb{R}^d$ 。

为什么这样做?

- **计算开销下降:** 直接把 $n_h \cdot c$ 投影到 d 会很大; 分组后先降维再拼接, 可以显著减少矩阵乘法量。
- **可控参数:** 中间维度 d_g 可以小于 $c \cdot n_h/g$, 节省参数。
- **保持多头信息:** 每组内部的多头信息被压缩, 中间输出再汇总到最终 hidden state, 保留了多头 attention 的语义。

2 混合注意力机制中的其他细节优化

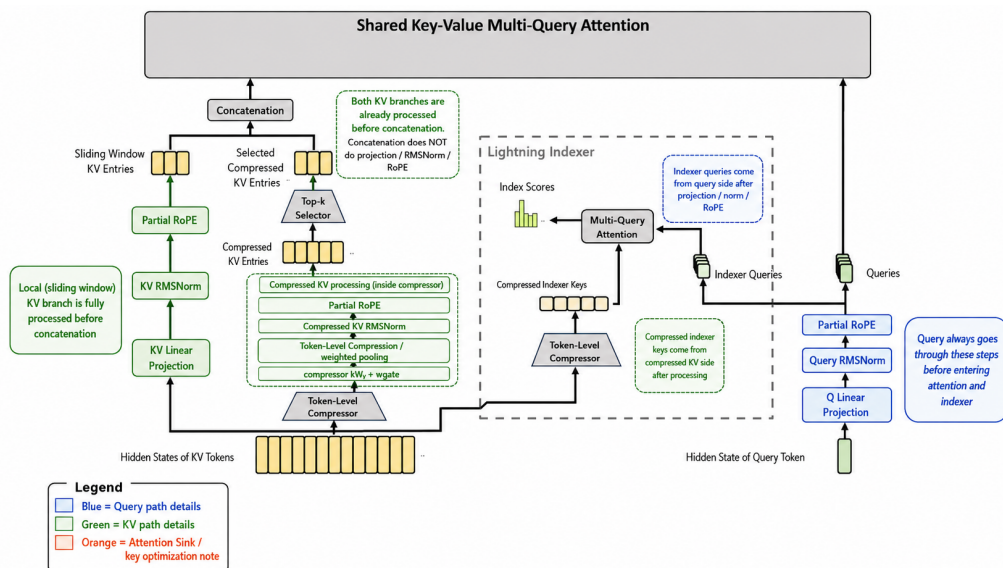


图 5: 混合注意力机制中的其他细节优化

2.1 Query 和 Key-Value 进入 Attention 前的归一化

基本上在每次**线性投影**（也就是线性层）之后都会做一下**RMSNorm 归一化**。老生常谈：归一化操作可以避免注意力 logit 爆炸，并有助于提升训练稳定性。做了这个之后，就不用再使用 **QK-Clip** 以裁剪的手段防止注意力 logits 爆炸了。

$$Q_i = \frac{\tilde{Q}_i}{\max(1, \|\tilde{Q}_i\|_2/c)}, \quad K_j = \frac{\tilde{K}_j}{\max(1, \|\tilde{K}_j\|_2/c)}$$

2.2 部分 Rotary 位置编码

Partial Rotary Positional Embedding

dsv4 对 Query、KV Entry 和注意力输出都使用了 **RoPE 位置编码**，但只加在**向量的最后 64 维**。

为什么只加到后 64 维？

RoPE 的目的是让模型捕捉**相对位置关系**，而非全部向量都承担 position encoding，能避免过度干扰内容表示。此外这样做不仅省，还可以使得前面的 **KV 缓**

存可以在不同位置被复用，还可以减少语义与位置信息的耦合。

对 Q、KV 进行 RoPE 操作是标准做法，但为什么也还要对注意力输出也做呢？

标准的 Transformer 结构里是对 Q、K 做 RoPE 操作并做点积，但 CSA 的设计里，KV entries 是同一个向量同时兼顾 Key 和 Value，也就是共享 KV，这使得原本不应该添加 RoPE 的 V 也有了和 K 一样的位置。

标准注意力公式：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

CSA 在乘 V 之前的操作和标准注意力没啥区别。但错就错在下一步，把 K_j （也就是 V）和 RoPE 表达展开：

$$K_j = R(j)K_j^{(0)}$$

可见这导致朴素的核心注意力输出 $\{o_{t,i}\}$ 将携带绝对位置编码 $R(j)$ 。

为了修正这一错误，对每个注意力输出 $o_{t,i}$ 的最后 64 个维度施加位置为 $-i$ 的 RoPE 变换。

这个 $-i$ 实际上就是 Q_i 的位置信息，因为本身 $Q_i^T K_j$ 点积结果就是：

$$Q_i^{(0)T} R(j-i)K_j^{(0)}$$

相当于给最后乘的 $V = K$ ，从 $R(j)K_j^{(0)}$ 变成 $R(j-i)K_j^{(0)}$ 。

通过这种方式，核心注意力的输出也将携带相对位置编码。虽然标准 Transformer 输出不显式写出位置编码词项 $\sum_j A_{i,j}V_j$ ，但其 attention 权重已经是相对位置感知，只不过没有把位置写进输出的向量本身。DeepSeek V4 修正输出是为了在 KV/V 共享且 KV entry 带位置时仍然保持 translation invariant。

为什么用 $R(-i)$ 而不是 $R(-j)$ 修正？

从表面来看，最后 V 多了位置 $R(j)$ ，直接用 $R(-j)$ 去抵消掉不就好了吗？这样就和原始 Transformer 位置信息一样了。但仔细推导：

$$Q'_i = R(i)Q_i, \quad K'_j = R(j)K_j$$

注意力权重是：

$$A_{i,j} \propto \exp(Q_i^T R(j-i)K_j)$$

如果是标准模型：

$$O_i = \sum_j A_{i,j} V_j$$

因为 V_j 没有经过 RoPE，这个输出只在权重上带相对位置信息。但 DeepSeek V4：

$$O_i = \sum_j A_{i,j} R(j) K_j$$

输出就含有 $R(j)$ 绝对位置成分。

这里用到了旋转矩阵的性质 $R(a)R(b) = R(a+b)$ 和 $R(-j) = R(j)^{-1}$ 。可选择的乘法操作只有两类： $R(-j)$ 与 $R(-i)$ 。因为当前输出不是单一某一个 j 的向量，而是多个不同项 j 累加的加权和，所以没办法直接在外边乘同一个 $R(-j)$ 。但是可以乘同一个 $R(-i)$ ，因为在一个给定查询位置 i 下，这个位置编码 $R(i)$ 对该 query 来说是固定的。它只与这个 query 的位置 i 有关，而 $R(j)$ 随着不同的 key 位置 j 而变化。

$$R(-i)O_i = \sum_j A_{i,j} R(j-i) K_j$$

2.3 Attention Sink

注意力 sink 技巧这一步已经到了注意力的输出。它为每个注意力头设置一组可学习的汇聚 logit：

$$\{z'_1, z'_2, \dots, z'_{n_h}\}$$

对于第 h 个注意力头， $\exp(z'_h)$ 被加入注意力分数的分母：

$$s_{h,i,j} = \frac{\exp(z_{h,i,j})}{\sum_k \exp(z_{h,i,k}) + \exp(z'_h)}$$

其中 $s_{h,i,j}, z_{h,i,j} \in \mathbb{R}$ 分别表示第 h 个注意力头中，第 i 个查询 Token 与第 j 个前驱 Token 或压缩块之间的注意力分数和注意力 logit。

这一技巧允许每个查询头将其总注意力分数调整为不等于 1，甚至接近 0。这个的好处是强化重要信息，抑制冗余。

参考

- DeepSeek v4 的架构创新优化.pdf
- figures 目录下的 CSA、HCA、分组投影与混合注意力细节配图